

Integrating AEM Experience Fragments (XFs) with Next.js/React is achieved by **treating AEM as a headless CMS** and consuming the XFs as **JSON data** via AEM's Content Services API or GraphQL API. The front-end Next.js application is responsible for fetching and rendering this data using its own React components. [🔗](#)

High-Level Steps for Integration [🔗](#)

The process involves both back-end AEM configuration and front-end Next.js development. [🔗](#)

1. AEM Configuration (Back-end) [🔗](#)

- **Create Experience Fragments:** In AEM, create the XFs using the standard page editor. These are groups of components (e.g., image, text, button) with defined layouts. They are typically stored under `/content/experience-fragments`.
- **Expose as JSON:** Ensure the XFs are accessible as JSON endpoints.
 - **Sling Model Exporter:** This is the standard method to expose content structure (including XFs) as JSON objects for external consumption.
 - **AEM GraphQL API:** For more complex content structures, the GraphQL API allows for more efficient and flexible data fetching, enabling you to request only the specific data needed.
- **Publish:** Publish the Experience Fragments to the AEM Publish instance so they are publicly accessible to your Next.js application. [🔗](#)

2. Next.js/React Implementation (Front-end)

- **Set up the Next.js Project:** Use the Adobe AEM Project Archetype to generate a project structure that is compatible with a remote SPA setup, or configure an existing Next.js app. You will typically remove the default front-end module in the archetype.
- **Fetch Data:** In your Next.js components, use data-fetching methods like `useEffect` (for client-side fetching), `getServerSideProps` (for Server-Side Rendering), or `getStaticProps` (for Static Site Generation) to call the AEM JSON or GraphQL endpoints.
 - **REST Example (using `fetch`):**

javascript [🔗](#)

```
useEffect(() => {  
  const fetchFragment = async () => {  
    const response = await fetch('https://your-aem-instance.com/path/to/experience-frag  
    const fragmentData = await response.json();
```

```
// set state or process data  
};  
fetchFragment();  
}, []);
```

- **GraphQL Example:** You can use libraries like Apollo Client to manage GraphQL queries efficiently.
- **Map and Render Components:** Map the received JSON structure from AEM to corresponding React components in your Next.js app. This ensures a flexible and component-based approach where AEM manages content, and React manages rendering logic. [🔗](#)

Key Considerations

- **Authoring Experience:** For an enhanced authoring experience, you can implement **in-context editing** using the **AEM Universal Editor** or the AEM SPA Editor SDK. This allows content authors to edit XF content directly within the visual context of the Next.js application preview.
- **Performance:** Next.js features like SSR and SSG are crucial for performance. Utilize caching strategies and CDNs to minimize API calls to AEM.
- **Security:** Secure your AEM API access using appropriate measures like OAuth or API keys, especially for author instances. [🔗](#)

This headless approach provides a clear separation of concerns, leveraging AEM's robust content management capabilities and Next.js's performance and modern front-end features